Building a Large Language Language Model

www.drshetty.in

Steps to Create a Large Language Model

Building a Large Language Model (LLM) is one of the most complex engineering tasks in the world today. A 5-step process is followed so that the LLM learns and models the relationships between ideas, words, and contexts

Step 1: Data Curation (The Foundation)

In artificial intelligence, the principle “Garbage In, Garbage Out” is foundational. The quality, diversity, and structure of training data directly determine the intelligence and reliability of the resulting model.

a. Collection and Scaling Laws:
   - Modern LLMs follow scaling laws governed by compute, dataset size, and parameters.
   - Training data is sourced from web crawls, academic papers, books, and code repositories.

b. Deduplication and Cleaning:
   - Exact deduplication uses SHA-1 or MD5 hashing.
   - Fuzzy deduplication uses MinHash or LSH.
   - Noise such as HTML tags and gibberish is removed.

Step 2: Tokenization (The Secret Code)

- Tokenization converts text into tokens mapped to numbers.
- Techniques include Byte Pair Encoding (BPE).
- Each token is converted into an embedding vector representing semantic meaning.

Step 3: Model Architecture (The Brain Design)

- LLMs use the Transformer architecture.
- Self-attention enables context awareness.
- Mixture of Experts activates only relevant sub-networks for efficiency

Step 4: Training at Scale (The Learning Phase)

1. Pre-training: Next-token prediction on large corpora.
2. Supervised Fine-Tuning: Human-written question–answer datasets.
3. RLHF: Reinforcement Learning from Human Feedback.

Step 5: Evaluation (The Final Examination)

- Benchmarks include
  a. MMLU
  b. HumanEval, and
  c. cosine similarity-based evaluations.

From RNNs to Transformers: Evolution of Language Modeling

- Earlier neural networks like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs) were the "old school" way of teaching computers to read.
- While they were a great start, they had massive flaws that stopped them from becoming as smart as today's AI.
- The Transformer (introduced in 2017) changed everything.
- Why the old models failed and how Transformers solved those problems?

1. The Sequential Processing Bottleneck

- **The Problem (RNN/LSTM):** Think of an RNN like a person reading a book one word at a time, moving their finger along the line. To understand word #50, they must read words 1 through 49 first.
  - Slow Training: You can't skip ahead, so you can't use all the power of modern computer chips (GPUs) at once.
  - High Latency: Long sentences take a long time to process because the model is stuck in a "waiting line.“

- **The Transformer Solution**: Transformers read the entire sentence all at once. It’s like taking a photo of a whole page instead of scanning it line by line.
  - Parallelization: Because it sees everything at once, it can use thousands of computer cores to process data simultaneously.

2. Long-Range Dependencies (The "Memory" Problem)

- **The Problem (RNN/LSTM):** Information in an RNN "fades" over time. If a sentence starts with "The Queen, who traveled through many lands and met many people, finally reached her...", the RNN might forget the word "Queen" by the time it gets to the end.
  - Context Loss: The model loses the "big picture" in long documents.

- **The Transformer Solution:** Self-Attention Transformers use a "Searchlight" called Self-Attention. Every word looks at every other word in the sentence at the same time.
  - Direct Connections: The last word in a 1,000-word essay has a direct "line of sight" to the first word. No information is lost, no matter the distance.

3. Vanishing and Exploding Gradients

- **The Problem (RNN/LSTM):** During training, the model learns by sending "feedback signals" (gradients) backward. In RNNs, this signal has to travel through every single word in the sequence.
  - Vanishing: The signal gets so weak it disappears (the model stops learning).
  - Exploding: The signal gets so huge it crashes the math (the model becomes unstable).

- **The Transformer Solution:** Transformers don't use "time steps" or loops. The signal travels through a constant, short path through the layers. This makes the math stable and allows us to build models with hundreds of layers without them "breaking."

4. Information Bottlenecks

- **The Problem (RNN/LSTM):** An RNN tries to squeeze the meaning of an entire sentence into one tiny "hidden state" (like trying to summarize a whole movie into a single sticky note).
  - Compression Loss: It struggles to remember multiple characters, complex grammar, or deep meanings all at once.

- **The Transformer Solution:** Instead of one sticky note, the Transformer keeps a detailed map for every single word. Each word maintains its own rich set of information about the words around it.

5. Scalability (Big Data & Big Models)

- **The Problem (RNN/LSTM):** Making an RNN 10x bigger doesn't necessarily make it 10x smarter; it just makes it 10x slower and harder to train. They hit a "ceiling" where more data didn't help much.

- **The Transformer Solution:** Transformers are "Scaling Monsters." The more data and more parameters you give them, the smarter they get. This scalability is exactly why we went from small translators to massive models like GPT-4.

Parameters in Large Language Models (LLMs)

- Parameters are the "knobs and dials" of an artificial intelligence model.
- They are the internal variables that the model learns and adjusts during the training process to map inputs to correct outputs.

1. The Mathematical Essence: Weights and Biases

- In a transformer-based LLM, parameters primarily consist of two components:
  - Weights (): These determine the strength of the connection between neurons across different layers. They multiply the input data to signify importance. For example, in the phrase "The knight rode the...", a high weight might be placed on the relationship between "knight" and "horse."
  - Biases (): these are additive values that allow the model to shift the activation function. They help the model represent patterns even when the input signal is weak or zero.
  - The fundamental operation for a single layer can be expressed as:

$$y = f(W \cdot x + b)$$

  - Where x is the input vector, W is the weight matrix, b is the bias vector, and f is a non-linear activation function.

2. What Parameters Actually Store

- Parameters do not store "facts" like a traditional database; instead, they store statistical relationships and conceptual abstractions.
  - Syntax and Grammar: Parameters encode the rules of language (e.g., that a subject is usually followed by a verb).
  - World Knowledge: Relationships between entities (e.g., "Paris" is related to "France" and "Eiffel Tower") are distributed across millions of parameters.
  - Reasoning Patterns: Parameters learn the logical flow of arguments or the steps required to solve a math problem.
  - Polysemy Resolution: They help the model distinguish between different meanings of the same word (e.g., a "bank" of a river vs. a "bank" for money) based on the surrounding tokens.

3. The Architecture of Parameters

- In an LLM, parameters are organized into specific functional blocks within the Transformer architecture:

Component	Role of Parameters
Embeddings	Map discrete tokens into high-dimensional vectors.
Attention Mechanisms	Determine which parts of the input sequence are most relevant to each other.
Feed-Forward Networks	Process the information gathered by attention to create higher-level abstractions.
Layer Normalization	Parameters (and) that keep the mathematical signal stable during deep processing.

4. Parameter Count and Scaling

- The "size" of a model (e.g., GPT-3 has 175 billion parameters) is a proxy for its capacity.
  - Emergent Abilities: Research suggests that once a model hits a certain parameter threshold, it "unlocks" new capabilities like zero-shot reasoning or coding skills that smaller models lack.
  - The Scaling Laws: Generally, increasing parameters, training data, and compute leads to better performance, though there are diminishing returns if the data quality is poor.
  - Compute Costs: More parameters mean the model requires more VRAM (Video RAM) to store the weights and more FLOPs (Floating Point Operations) to perform calculations during inference.

5. How Parameters Change: Training vs. Inference

- Training (Learning): Parameters start as random numbers. Through backpropagation and Gradient Descent, the model calculates its error (loss) and updates the parameters to be slightly more accurate.
- Inference (Using): Once training is complete, the parameters are "frozen." When you prompt the model, the data flows through these fixed numerical values to produce an answer.
- Fine-Tuning: A pre-trained model can have its parameters slightly adjusted on a smaller, specific dataset to specialize in a task (like medical advice or legal drafting).

Note: While "more parameters" often means "smarter," it is not the only factor. The quality of the training data and the efficiency of the architecture are just as critical. A well-trained 7B parameter model can often outperform a poorly trained 30B model.